

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería



Proyecto #2 - Avances

Norman Adonay Aguirre Lepe

Erick Marroquín

Base de Datos 1

Guatemala, 2026

Índice

Descripción del Programa	3
Identificación de Entidades y Atributos	4
Identificación de Claves Candidatas y Clave Primaria	6
Dependencias Funcionales	7
Normalización	9
Forma No Normal (FNN)	9
Violaciones detectadas (grupos repetitivos).....	9
Primera Forma Normal (1FN)	10
Justificación técnica	10
Descomposición:.....	10
Segunda Forma Normal (2FN)	11
Justificación técnica	11
Tabla con violación: DETALLE_VENTA_TEMP.....	11
Descomposición a 2FN	11
Tabla con violación: DETALLE_COMPRA_TEMP	12
Descomposición a 2FN	12
Tercera Forma Normal (3FN).....	12
Justificación técnica	12
Tabla con violación: PRODUCTO_TEMP.....	12
Descomposición a 3FN	12
Tabla con violación: USUARIO_TEMP.....	13
Descomposición a 3FN:	13
Resultado Final de la Normalización	13
Diagrama Entidad Relación.....	16

Descripción del Programa

BrickLand Store es una tienda especializada en productos LEGO orientada a niños, coleccionistas y entusiastas. El presente proyecto consiste en el diseño e implementación de una aplicación web para gestionar el inventario y las ventas de la tienda. La aplicación integra un frontend, un backend con SQL explícito y una base de datos relacional PostgreSQL, desplegada mediante Docker.

El dominio del negocio centraliza la operación completa de BrickLand Store: manejo de productos LEGO agrupados en categorías temáticas (City, Star Wars, Technic, Harry Potter, Marvel/DC, Creator, Accesorios), adquisición de mercadería a través de proveedores autorizados, registro de clientes, atención de ventas por parte de empleados (cajeros y vendedores), y control de stock en tiempo real.

El sistema permitirá:

- Registrar y gestionar el catálogo de sets LEGO y accesorios por categoría temática y proveedor.
- Controlar el stock disponible y emitir alertas de bajo inventario.
- Registrar clientes (padres de familia, coleccionistas, aficionados) con nombre, correo y teléfono.
- Registrar empleados de la tienda (cajeros y vendedores) con sus datos laborales.
- Procesar ventas: seleccionar productos, calcular totales, descontar stock y guardar el detalle.
- Registrar compras a proveedores para actualizar el inventario.
- Generar reportes de ventas por fecha, productos más vendidos, ingresos totales e inventario disponible.
- Autenticar usuarios (empleados) con roles diferenciados (admin, vendedor/cajero).
- Exportar reportes a CSV o PDF desde la interfaz web.

El alcance abarca desde el diseño de la base de datos normalizada hasta la implementación completa de la aplicación web, incluyendo CRUD de entidades, consultas avanzadas (JOIN, subqueries, CTEs, VIEWS), transacciones explícitas con manejo de errores, y funcionalidades avanzadas de autenticación y exportación.

Identificación de Entidades y Atributos

CATEGORIA

Agrupar los sets LEGO por temática (City, Star Wars, Technic, Harry Potter, Marvel/DC, Creator, Accesorios).

id_categoria, nombre, descripción

PROVEEDOR

Empresas que suministran los sets LEGO a BrickLand Store (ej. LEGO Guatemala Distribuidor Oficial, Importadora BrickWorld, ToyHouse Supply Co.).

id_proveedor, nombre, contacto, telefono, email, dirección

PRODUCTO

Sets LEGO y accesorios disponibles en la tienda (ej. Millennium Falcon, Hogwarts Castle, Pack de Minifiguras). Pertenece a una categoría y es suministrado por un proveedor.

id_producto, nombre, descripcion, precio_unitario, stock, stock_minimo, id_categoria, id_proveedor

EMPLEADO

Cajeros y vendedores encargados de registrar las ventas en BrickLand Store.

id_empleado, nombre, apellido, cargo, telefono, email, fecha_contratacion, activo

CLIENTE

Padres de familia, niños, coleccionistas y aficionados que realizan compras en la tienda.

id_cliente, nombre, apellido, telefono, email, nit, fecha_registro

VENTA

Transacción registrada por un empleado para un cliente. El sistema calcula el total y descuenta el stock al confirmarla.

id_venta, fecha, total, estado, id_cliente, id_empleado

DETALLE_VENTA

Línea de detalle de una venta: qué producto, en qué cantidad y a qué precio fue vendido.

id_detalle, id_venta, id_producto, cantidad, precio_unitario, subtotal

COMPRA

Orden de reabastecimiento realizada a un proveedor, gestionada por un empleado.

id_compra, fecha, total, estado, id_proveedor, id_empleado

DETALLE_COMPRA

Línea de detalle de una compra a proveedor: qué producto, en qué cantidad y a qué costo fue adquirido.

id_detalle, id_compra, id_producto, cantidad, costo_unitario, subtotal

USUARIO

Credenciales de acceso al sistema. Cada usuario está vinculado a un empleado y tiene un rol (admin o vendedor).

id_usuario, username, password_hash, rol, activo, id_empleado

Identificación de Claves Candidatas y Clave Primaria

Entidad	Claves candidatas	Clave primaria seleccionada
CATEGORIA	id_categoria, nombre	id_categoria
PROVEEDOR	id_proveedor, email	id_proveedor
PRODUCTO	id_producto, nombre	id_producto
EMPLEADO	id_empleado, email	id_empleado
CLIENTE	id_cliente, nit, email	id_cliente
VENTA	id_venta	id_venta
DETALLE_VENTA	id_detalle, (id_venta, id_producto)	id_detalle (surrogada)
COMPRA	id_compra	id_compra
DETALLE_COMPRA	id_detalle, (id_compra, id_producto)	id_detalle (surrogada)
USUARIO	id_usuario, username, id_empleado	id_usuario

Dependencias Funcionales

CATEGORIA

id_categoria → nombre, descripcion

nombre → id_categoria (clave candidata)

PROVEEDOR

id_proveedor → nombre, contacto, telefono, email, direccion

email → id_proveedor (clave candidata)

PRODUCTO

id_producto → nombre, descripcion, precio_unitario, stock, stock_minimo, id_categoria, id_proveedor

EMPLEADO

id_empleado → nombre, apellido, cargo, telefono, email, fecha_contratacion, activo

email → id_empleado (clave candidata)

CLIENTE

id_cliente → nombre, apellido, telefono, email, nit, fecha_registro

nit → id_cliente (clave candidata)

email → id_cliente (clave candidata)

VENTA

id_venta → fecha, total, estado, id_cliente, id_empleado

DETALLE_VENTA

id_detalle → id_venta, id_producto, cantidad, precio_unitario, subtotal

(id_venta,id_producto) → cantidad, precio_unitario, subtotal (clave candidata compuesta)

COMPRA

id_compra → fecha, total, estado, id_proveedor, id_empleado

DETALLE_COMPRA

id_detalle → id_compra, id_producto, cantidad, costo_unitario, subtotal

(id_compra,id_producto) → cantidad, costo_unitario, subtotal (clave candidata compuesta)

USUARIO

id_usuario → username, password_hash, rol, activo, id_empleado

username → id_usuario (clave candidata)

id_empleado → id_usuario (clave candidata)

Normalización

Forma No Normal (FNN)

Se parte de una sola tabla que agrupa todos los datos del negocio de BrickLand Store:

BRICKLAND(

id_venta, fecha_venta, total_venta, estado_venta,

id_cliente, nombre_cliente, apellido_cliente, telefono_cliente,

email_cliente, nit_cliente,

id_empleado, nombre_empleado, apellido_empleado, cargo_empleado,

email_empleado,

id_producto, nombre_producto, descripcion_producto, precio_unitario,

stock, stock_minimo,

id_categoria, nombre_categoria, descripcion_categoria,

id_proveedor, nombre_proveedor, contacto_proveedor,

telefono_proveedor, email_proveedor,

cantidad_vendida, subtotal_detalle,

id_compra, fecha_compra, total_compra, cantidad_comprada, costo_unitario,

username, password_hash, rol_usuario

)

Violaciones detectadas (grupos repetitivos)

Una venta puede incluir múltiples sets LEGO, y un producto puede aparecer en múltiples ventas y compras. Esto genera dependencias multivaluadas:

id_venta → id_producto, cantidad_vendida (multivaluado)

id_venta → id_compra (multivaluado)

id_producto → id_proveedor, id_categoria (mezclado con ventas)

Esto produce redundancia masiva, anomalías de inserción (no se puede registrar un set LEGO sin una venta), anomalías de eliminación (borrar una venta destruye datos del producto) y anomalías de actualización (cambiar el precio del Millennium Falcon requiere modificar miles de filas).

Primera Forma Normal (1FN)

Justificación técnica

La tabla BRICKLAND no cumple 1FN porque contiene grupos repetitivos (un registro por venta se repite por cada set LEGO vendido) y atributos multivaluados. Se elimina toda repetición asignando una PK a cada entidad independiente.

Descomposición:

CATEGORIA_TEMP: id_categoria, nombre_categoria, descripcion_categoria

PROVEEDOR_TEMP: id_proveedor, nombre_proveedor, contacto_proveedor,
telefono_proveedor, email_proveedor

PRODUCTO_TEMP: id_producto, nombre_producto, descripcion_producto,
precio_unitario, stock, stock_minimo, id_categoria, id_proveedor

EMPLEADO_TEMP: id_empleado, nombre_empleado, apellido_empleado,
cargo_empleado, telefono_empleado, email_empleado, fecha_contratacion, activo

CLIENTE_TEMP: id_cliente, nombre_cliente, apellido_cliente, telefono_cliente,
email_cliente, nit_cliente

VENTA_TEMP: id_venta, fecha_venta, total_venta, estado_venta, id_cliente,
id_empleado

DETALLE_VENTA_TEMP: id_venta, id_producto, cantidad, precio_unitario, subtotal

COMPRA_TEMP: id_compra, fecha_compra, total_compra, estado_compra, id_proveedor, id_empleado

DETALLE_COMPRA_TEMP: id_compra, id_producto, cantidad, costo_unitario, subtotal)

USUARIO_TEMP: id_usuario, username, password_hash, rol, activo, id_empleado

Segunda Forma Normal (2FN)

Justificación técnica

La 2FN exige que ningún atributo no-clave dependa de una parte de una clave primaria compuesta. Se analizan las tablas con PK compuesta candidata.

Tabla con violación: DETALLE_VENTA_TEMP

La PK compuesta candidata natural es: id_venta, id_producto. Se detecta la siguiente dependencia parcial:

id_producto → precio_unitario

precio_unitario depende únicamente de id_producto, no de la combinación (id_venta, id_producto).

Descomposición a 2FN

DETALLE_VENTA

id_detalle, ←PK surrogada para eliminar la dependencia parcial

id_venta,

id_producto,

cantidad,

precio_unitario, ← precio historico al momento de la venta

subtotal

Tabla con violación: DETALLE_COMPRA_TEMP

La PK compuesta candidata (id_compra, id_producto) expone:

id_producto → costo_unitario (costo de catalogo del proveedor, dependencia parcial)

Descomposición a 2FN

DETALLE_COMPRA

id_detalle,

id_compra,

id_producto,

cantidad,

costo_unitario, ← costo historico al momento de la compra

subtotal

Las demás tablas (VENTA_TEMP, CLIENTE_TEMP, EMPLEADO_TEMP, etc.) tienen PK simples, por lo que no pueden tener dependencias parciales y ya cumplen 2FN sin modificación.

Tercera Forma Normal (3FN)

Justificación técnica

La 3FN exige que ningún atributo no-clave dependa de otro atributo no-clave (elimina dependencias transitivas).

Tabla con violación: PRODUCTO_TEMP

Se detectan las siguientes cadenas transitivas:

id_producto → id_categoria → nombre_categoria, descripcion_categoria

id_producto → id_proveedor → nombre_proveedor, contacto_proveedor,

Descomposición a 3FN

CATEGORIA: id_categoria, nombre, descripcion

PROVEEDOR: id_proveedor, nombre, contacto, telefono, email, direccion

PRODUCTO: id_producto, nombre, descripcion, precio_unitario, stock, stock_minimo,
id_categoria, ←FK, no arrastra atributos de CATEGORIA
id_proveedor ← FK, no arrastra atributos de PROVEEDOR

Tabla con violación: USUARIO_TEMP

Se detecta la siguiente cadena transitiva:

id_usuario → id_empleado → nombre_empleado, cargo_empleado, email_empleado,

Los datos laborales del empleado (nombre, cargo, teléfono) dependen de id_empleado, no de id_usuario. Si un cajero cambia de nombre, ese dato se actualizaría en EMPLEADO sin tocar USUARIO.

Descomposición a 3FN:

EMPLEADO(id_empleado, nombre, apellido, cargo, telefono, email,
fecha_contratacion, activo)

USUARIO(id_usuario, username, password_hash, rol, activo,
id_empleado) <- FK, no arrastra atributos de EMPLEADO

Las tablas CLIENTE_TEMP, VENTA_TEMP, COMPRA_TEMP, DETALLE_VENTA y DETALLE_COMPRA no presentan dependencias transitivas: todos sus atributos no-clave dependen directamente de su PK.

Resultado Final de la Normalización

CATEGORIA(id_categoria, nombre, descripcion)

PK: id_categoria

PROVEEDOR(id_proveedor, nombre, contacto, telefono, email, direccion)

PK: id_proveedor

PRODUCTO(id_producto, nombre, descripcion, precio_unitario, stock,
stock_minimo, id_categoria, id_proveedor)

PK: id_producto

FK: id_categoria -> CATEGORIA(id_categoria)

FK: id_proveedor -> PROVEEDOR(id_proveedor)

EMPLEADO(id_empleado, nombre, apellido, cargo, telefono, email,
fecha_contratacion, activo)

PK: id_empleado

CLIENTE(id_cliente, nombre, apellido, telefono, email, nit, fecha_registro)

PK: id_cliente

VENTA(id_venta, fecha, total, estado, id_cliente, id_empleado)

PK: id_venta

FK: id_cliente -> CLIENTE(id_cliente)

FK: id_empleado -> EMPLEADO(id_empleado)

DETALLE_VENTA(id_detalle, id_venta, id_producto, cantidad,
precio_unitario, subtotal)

PK: id_detalle

FK: id_venta -> VENTA(id_venta)

FK: id_producto -> PRODUCTO(id_producto)

COMPRA(id_compra, fecha, total, estado, id_proveedor, id_empleado)

PK: id_compra

FK: id_proveedor -> PROVEEDOR(id_proveedor)

FK: id_empleado -> EMPLEADO(id_empleado)

DETALLE_COMPRA(id_detalle, id_compra, id_producto, cantidad,
costo_unitario, subtotal)

PK: id_detalle

FK: id_compra -> COMPRA(id_compra)

FK: id_producto -> PRODUCTO(id_producto)

USUARIO(id_usuario, username, password_hash, rol, activo, id_empleado)

PK: id_usuario

FK: id_empleado -> EMPLEADO(id_empleado)

UNIQUE: username

UNIQUE: id_empleado

Diagrama Entidad Relación

